Proc. of the International Conference on Electrical, Computer and Energy Technologies (ICECET)
9-10 December 2021, Cape Town-South Africa

A Comparison of Multiple Markov Chains Algorithms for Bayesian Updating

Marwan Sherri
Department of Mechanical Engineering Science
University of Johannesburg
Johannesburg, South Africa
rfmsheri@yahoo.com

Ilyes Boulkaibet
Electrical Engineering Department
American University of the Middle East
Kuwait City, Kuwait
ilyes.boulkaibet@aum.edu.kw

Tshilidzi Marwala
Institute of Intelligent Systems
University of Johannesburg
Johannesburg, South Africa
tmarwala@uj.ac.za

Michael Friswell
College of Engineering
Swansea University
Swansea, United Kingdom
m.i.friswell@swansea.ac.uk

2021 International Conference on Electrical, Computer and Energy Technologies (ICECET) | 978-1-6654-4231-2/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICECET52533.2021.9698446

***Abstract*— In this paper, three advanced multiple Markov chains algorithms are compared for Bayesian model updating problems. The algorithms, namely, the Differential Evolution Markov Chain (DE-MC), the Differential Evolution Markov Chain with snooker update (DE-MCS), and the Population Markov Chain Monte Carlo (Pop-MCMC), are advanced versions of evolutionary techniques that utilize the multi-chain mechanism to approximate the posterior Probability Density Function (PDF). This paper examines these algorithms to solve the Finite Element Model Updating (FEMU) problem based on the Bayesian approach. FEMU is an optimization problem that can be applied in structural dynamics to increase the correlations between the modelled structure using the Finite Element Method (FEM) and the experiment data. Furthermore, the associated uncertainties of the modelled structure are can also be obtained using Bayesian inference where the posterior PDF is used to describe the uncertain parameters of the FE model. This paper addresses the efficiency and the performance of the algorithms to solve the same Bayesian updating problem. The algorithms are detailed and introduced for the FEMU problem. Then, the three procedures are employed to update the same structural example with real data. The advantages and the limitations of each method are discussed. The obtained results are analysed and compared, while the performance of each algorithm is explained in detail and the optimum updating model will be highlighted.**

Keywords*— *Bayesian model updating; Markov Chain Monte Carlo; differential evolution; finite element model; snooker updater; population Markov Chain Monte Carlo; evolutionary algorithm.

I. INTRODUCTION

Finite Element Methods (FEMs) [1] are numerical tools for engineering design and simulation. In structural engineering, FEMs can accurately predict the system behavior when applied for a simple structural analysis. In contrast, the FEM loses its efficiency when used to simulate a complex structural system. Therefore, the initial FE model must be updated to increase the correlation between the analytical solutions and the actual system response. Finite Element Model Updating (FEMU) [2] are optimization techniques used to minimize the differences between the FE models and the modal data. Two main approaches are known for FEMU, namely (i) the direct updating methods, and (ii) the indirect updating methods. Direct updating techniques are sufficient to update the structural model and to minimize the errors between the actual and the analytical data. However, these methods may produce non-realistic updating values, such as a zero-mass quantity or a negative elastic value. Thus, the direct updating methods are not preferable when several updating elements are considered in the structural analysis. The indirect updating techniques, also called iterative updating methods, are more computationally efficient, accurate, and guaranteed to compute updating results with physically meaningful values.

Bayesian methods are popular for quantifying the uncertainties that are incorporated with the system parameters. In Bayesian theory, the uncertainties of the modelled parameters are characterized through a posterior Probability Density Function (PDF) [3, 4]. The function defines the updating parameters as random variables. Unfortunately, the analytical solutions for this function cannot be obtained because of the complexity and the high-dimensionally of this function. Alternatively, sampling methods are used to approximate solutions for complex distributions. Among these methods, the Markov Chain Monte Carlo (MCMC) [5], represented by the standard Metropolis-Hastings (M-H) algorithm [3], is the widest accepted approach to draw samples from complex probability distributions. The acquired samples (solutions) are then used to approximate the posterior PDF, thereby updating the required structure. However, MCMC methods are limited when several updating parameters are involved and when the structure under consideration is sufficiently complex. For these reasons, further updating methods are proposed to be merged with the MCMC procedure to improve the quality of the sampling proposition.

The Evolutionary Algorithms (EAs) are promising for optimization problems [6]. Generally, the evolution-based search algorithm utilizes the current solutions' availability, known as the population, to produce potentially more accurate solutions. At each step (iteration), the ongoing population is modified through two common genetic operators (mutation and crossover) to produce the next generation. Each solution, known as a chromosome, is evaluated by a fitness function. This function is defined as the objective function of the optimization problem. The estimation of the function reflects the desire for the solution which the individual provides. The evolutionary paradigm is implemented through different classes, and the most

978-1-6654-4231-2/21/$31.00 ©2021 IEEE

common is the genetic algorithm. However, other advanced EAs, which are combined with the M-H sampler, is given by this paper.

In this paper, three advanced evolutionary-based algorithms are combined with the MCMC procedure and applied to sample from the posterior PDF. This combination is proposed to improve updating methods that can scale well with system complexity, provide better solutions, and avoid convergence towards a local optimum. The evolutionary Markov chain algorithms (multi-chain methods) generate the new chain by adopting the experience of the existing chains, which creates a desirable alternative solution to the single-chain method (i.e., the M-H algorithm) that adopts the random walk approach and uses only the information of the previous single move to propose the new chain. The presented algorithms in this paper are: (i) the Differential Evolution Markov Chain (DE-MC) algorithm, (ii) the Differential Evolution Markov Chain with Snooker update (DE-MCS), and (iii) the Population Markov Chain Monte Carlo (Pop-MCMC). These algorithms are evolutionary classes that are merged with the M-H method. The proposed samples are acquired in each method by a certain evolutionary strategy, and the M-H acceptance rule is applied to preserve the detailed balance condition.

The following sections detail the three algorithms and explain their approaches to host the M-H rule. The implementation of the algorithms to sample from the posterior PDF of the Bayesian statistics is given. The algorithms are used to solve the same structural updating problem with measured data. This paper aims to compare the three updating methods and show the best updating performer. The algorithms are compared by their efficiency (the solutions it provides), the sampling performance, the computational cost, and the convergence rate. The following section briefly describes Bayesian inference for the FEMU problem. Section 3 details the multiple Markov chains algorithms. Section 4 presents the application and the discussion of the model updating problem. Finally, Section 5 concludes the work.

II. Bayesian method

The uncertain parameters are defined as a stochastic vector and determined by Bayes' theorem [7, 8]:

$$P(\boldsymbol{\theta}|\mathcal{D},\mathcal{M}) \propto P(\mathcal{D}|\boldsymbol{\theta},\mathcal{M})\ P(\boldsymbol{\theta}|\mathcal{M}) \tag{1}$$

where $\boldsymbol{\theta}$ represents the vector of updating parameters, $\boldsymbol{\theta} \in \boldsymbol{\Theta} \subset \mathcal{R}^d$. $\mathcal{D}$ is the measured response of the structure, which is given by the natural frequencies f_i^m and mode shapes $\boldsymbol{\phi}_i^m$. $\mathcal{M}$ is the model class of the system and is defined by a different set of updating parameters for each class. $P(\boldsymbol{\theta}|\mathcal{M})$ is to the prior PDF that provides the known data about the uncertain parameters $\boldsymbol{\theta}$ given the model class $\mathcal{M}$ and with the absence of the measured data $\mathcal{D}$. $P(\mathcal{D}|\boldsymbol{\theta},\mathcal{M})$ is the likelihood function that gives the difference between the experimental data and the analytical results. $P(\boldsymbol{\theta}|\mathcal{D},\mathcal{M})$ refers to the posterior PDF of the updating parameters given $\mathcal{M}$ and $\mathcal{D}$. In this paper, only one model class is considered; thereby, $\mathcal{M}$ is removed for simplicity.

The final form of the posterior PDF $P(\boldsymbol{\theta}|\mathcal{D})$ of the updating parameters $\boldsymbol{\theta}$ given the modal data $\mathcal{D}$ is:

$$P(\boldsymbol{\theta}|\mathcal{D}) \propto \frac{1}{Z_s(\alpha,\beta_c)} \exp\left(-\frac{\beta_c}{2}\sum_i^{N_m}\left(\frac{f_i^m - f_i}{f_i^m}\right)^2 - \sum_i^Q \frac{\alpha_i}{2}\left\|\theta^i - \theta_0^i\right\|^2\right) \tag{2}$$

where N_m is the number of measured modes, and β_c is an arbitrary constant. f_i^m and f_i are the ith measured and analytical natural frequencies. Q is the number of the uncertain parameters, $\boldsymbol{\theta}_0$ gives the mean value of the uncertain parameters, α_i is the coefficient of uncertain parameters, $i = 1, \dots, Q$, and the Euclidean norm of $*$ is noted by $\|*\|$.

Eq. (2) presents the posterior PDF, representing the objective function for evolutionary optimization methods. For more details about the derivation of the prior function, the likelihood function, and the normalizing function $Z_s(\alpha, \beta_c)$ please see [3]. The complexity of the posterior PDF is increased when more updated parameters (three or above) are modelled. The accuracy of the updated results may decline when using a single-chain approach to search through the bounds of the parameter space. Thus, the advanced evolutionary Markov chain algorithms, which are proposed in this paper, are expected to overcome those limitations. The same FEMU case is tested independently for all algorithms. The following section explains and details the algorithms for the Bayesian FEMU problem.

III. Multiple Markov chains algorithms

A. DE-MC algorithm

Let $\boldsymbol{\theta}$ represent the d-dimensional vector of the uncertain parameters, $\boldsymbol{\theta} = \{\theta_1, \theta_2, \dots, \theta_d\}$. Each uncertain parameter has N chains, where $\theta_{(d)} = \{\theta_1, \theta_2, \dots, \theta_N\}$. This algorithm usually uses a large number of chains $N >> d$. However, all chains together represent the population that is evolved through the sampling process (new generation creation). The first population is generated by the prior function. The new sampling state $\boldsymbol{\theta}^*$ of the ith chain, $i \in \{1,2, \dots, N\}$, is computed as follows [9, 10]:

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_i + \gamma(\boldsymbol{\theta}_b - \boldsymbol{\theta}_a) + \varepsilon \tag{3}$$

$\boldsymbol{\theta}_a$ and $\boldsymbol{\theta}_b$ are two chains (i.e., updating vectors) picked at random, which should be different vectors, i.e. $\boldsymbol{\theta}_i \neq \boldsymbol{\theta}_a \neq \boldsymbol{\theta}_b$. γ is a tuning factor that controls the move distribution. The default setting for γ is $2.38/\sqrt{2d}$, which offers better mixing for the multiple operating chains and enhances the acceptance probability. ε is a minimal quantity and added to the new chain to prevent the degeneracy problem. Its value is drawn from a normal distribution with mean $\mu = 0$, and small variance σ^2, i.e., $\varepsilon \sim \mathrm{N}(0, \sigma^2)$. The proposed chain $\boldsymbol{\theta}^*$ will be either accepted or rejected by the Metropolis-Hastings acceptance rule:

$$r = \min\left\{1, \frac{P(\boldsymbol{\theta}^*|\mathcal{D})}{P(\boldsymbol{\theta}_i|\mathcal{D})}\right\} \tag{4}$$

B. DE-MCS algorithm

The DE-MCS algorithm [11] is an improved version of the DE-MC method and requires smaller number chains (e.g. $N_s = 3$). The difference vector $(\boldsymbol{\theta}_b - \boldsymbol{\theta}_a)$ is estimated from the past stored chains instead of considering only the current population as the DE-MC procedure. This mechanism reduces the chains which are needed to propose the new samples $\boldsymbol{\theta}^*$, since all the past chains are utilized. The three main advantages of the DE-

MCS algorithm, due to the fewer chains, are as follows. First, to speed up the convergence rate since only a few chains are required to move to the region of high-dense solutions. Second, outliers are minimum for fewer chains, since most of the chains are expected to converge to the area of solutions. Third, the real-time processing required to compute the updating problem is shorter when a smaller number of chains are evolved.

DE-MCS works as follows: let $\mathbf{\Theta}$ denote an $N_s \times d$ matrix that contains the states of the chains at the current iteration, and $\mathbf{M}$ is the matrix that preserves the current and the past state of the chains. The initial number of rows in $\mathbf{M}$ is defined by R_0, which has a default set $R_0 = 10d$, where that set provides a computational effective initial size of the matrix. At each generation (iteration), the updated chains (N_s rows) of $\mathbf{\Theta}$ are appended to $\mathbf{M}$, which expands the size of $\mathbf{M}$ through time t by $R + N_s$. Therefore, a thinning rate Z is added to reduce the storage capacity, which decreases the change of $\mathbf{M}$ by order $N_s/R = Z/t$. The proposed chain $\boldsymbol{\theta}_{\mathrm{i}}^*$ is generated by randomly selecting two other chains, $\boldsymbol{\theta}_{\mathrm{M}}^a$ and $\boldsymbol{\theta}_{\mathrm{M}}^b$, then projecting them orthogonally on the reference line ($\boldsymbol{\theta}_{\mathrm{M}}^{Pa}$ and $\boldsymbol{\theta}_{\mathrm{M}}^{Pb}$) and adding the scaling factor γ_s multiplied by the difference between the projection chains $\boldsymbol{\theta}_{\mathrm{M}}^{Pa}$ and $\boldsymbol{\theta}_{\mathrm{M}}^{Pb}$ to $\boldsymbol{\theta}_i$.

The proposal generation of the new chain $\boldsymbol{\theta}_{\mathrm{i}}^*$ by DE-MCS is given as:

$$\boldsymbol{\theta}_{\mathrm{i}}^* = \boldsymbol{\theta}_i + \gamma_s\left(\boldsymbol{\theta}_{\mathrm{M}}^{Pa} - \boldsymbol{\theta}_{\mathrm{M}}^{Pb}\right) \tag{5}$$

The default setting of the DE-MCS parameters are $N_s = 3$, $R_0 = 10d$, $Z = 10$, and $\gamma_s = 2.38/\sqrt{2d}$. The scaling factor γ_s uses $d = 1$ for all d, thus $\gamma_s = 2.38/\sqrt{2} \approx 1.7$. The reason for this choice ($d = 1$) is that the projection step minimizes the variance of the difference, which adjusts the resulting dimensionality. This default set returns a better acceptance rate. The generated sample $\boldsymbol{\theta}_{\mathrm{i}}^*$ is accepted by the Metropolis ratio:

$$r_s = min\left\{1, \frac{P(\boldsymbol{\theta}_{\mathrm{i}}^*|\mathcal{D})\,\|\boldsymbol{\theta}_{\mathrm{i}}^* - \boldsymbol{\theta}_{\mathrm{M}}^x\|^{d-1}}{P(\boldsymbol{\theta}_i|\mathcal{D})\,\|\boldsymbol{\theta}_i - \boldsymbol{\theta}_{\mathrm{M}}^x\|^{d-1}}\right\} \tag{6}$$

C. Pop-MCMC algorithm

The Pop-MCMC algorithm [12, 13] is a multichain stochastic algorithm for optimization and learning problems. This algorithm uses a population of solutions (multichain) to propose new candidates and later apply the Metropolis-Hastings (M-H) acceptance-rejection criterion. The algorithm employs advanced genetic operators (adaptive mutation and probabilistic crossover) for exploring and exploiting. Let $\widehat{\boldsymbol{\theta}} = \{\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots, \boldsymbol{\theta}_i, \dots, \boldsymbol{\theta}_N\}$ represent the population of the updating parameters, where $\boldsymbol{\theta}_i = \{\theta_i^1, \theta_i^2, \dots, \theta_i^Q\}$ is the ith chain, Q is the dimension of the uncertain parameters vector (known as an individual), and N is the number of the Markov chains. The adaptive mutation uses subspace sampling and outliers correction to accelerate the convergence. The subspace sampling is implemented through random updating to the chosen dimensions of the updating parameter vector $\boldsymbol{\theta}^i$ at each iteration. While the jump rate $\delta\boldsymbol{\theta}^i$ of the i th chain, $i = \{1,2,\dots,N\}$ at iteration $t = \{2, \dots, T\}$ is calculated from the chain series $\boldsymbol{\theta} = \{\theta_{t-1}^1, \dots, \theta_{t-1}^N\}$ due to [14]:

$$\delta\theta_V^i = \zeta_{\tilde{d}} + (1_{\tilde{d}} + \lambda_{\tilde{d}})\,\gamma_{(e,\tilde{d})} \sum_{j=1}^{e}\left(\theta_V^{a_j} - \theta_V^{b_j}\right) \tag{7}$$

where V is a $\tilde{d}$-dimensional subset of the updating parameter space, $\mathbb{R}^{\tilde{d}} \subseteq \mathbb{R}^Q$, $\tilde{d}$-entities are constructed with the help of the probabilistic crossover, which is discussed below, and $\delta\theta_{\neq V}^i = 0$. The tuning factor is $\gamma = \frac{2.38}{\sqrt{2e\tilde{d}}}$, and e is the selected number of the chain pairs, and its default value is $e = 3$. The vectors a and b contain e set values and are drawn from $\{1, \dots, i, .., N\}$. The values of λ and ζ are sampled independently from the multivariate uniform distribution $\mho_{\tilde{d}}(-\mu, \mu)$ and the normal distribution $\mathcal{N}_{\tilde{d}}(0, \bar{\mu})$, respectively. The value of μ is normally considered as a minimum such that $\mu = 0.1$, and $\bar{\mu} = 10^{-6}$ is small compared to the width of the target distribution. The mutation of the new sample $\boldsymbol{\theta}^*$ of the ith chain is:

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}^i + \delta\boldsymbol{\theta}^i \tag{8}$$

and the M-H acceptance probability $A(\boldsymbol{\theta}^*|\boldsymbol{\theta}^{k=i})$ is used to determine whether to accept this candidate or not:

$$(\boldsymbol{\theta}^*|\boldsymbol{\theta}^k) = min\left\{1, \frac{P(\boldsymbol{\theta}^*|D)\,q(\boldsymbol{\theta}^k|\boldsymbol{\theta}^*)}{P(\boldsymbol{\theta}^k|D)q(\boldsymbol{\theta}^*|\theta^k)}\right\} \tag{9}$$

Here $q(\boldsymbol{\theta}^k|\boldsymbol{\theta}^*)$ is the proposal distribution.

The algorithm adopts the probabilistic operator to exchange the information between the multichain in use. This step is applied before generating the candidate. An initial value of crossover $\mathcal{CR}$ is sampled from a geometric sequence $m_{\mathcal{CR}}$ such that $\mathcal{CR} = \left\{\frac{1}{m_{\mathcal{CR}}}, \frac{2}{m_{\mathcal{CR}}}, \dots, 1\right\}$ using the discrete multinomial distribution $\mathcal{M}(\mathcal{CR}, P_{\mathcal{CR}})$, where $P_{\mathcal{CR}}$ is the selection crossover probability. The default value of $m_{\mathcal{CR}}$ is 3. Then, the Q-vector $\mathbb{Z} = \{\mathbb{Z}_1, \dots, \mathbb{Z}_Q\}$ is drawn from the standard multivariate uniform distribution $\mathbb{Z} \sim U_Q(0,1)$. The values of j which satisfy $\mathbb{Z}_j \leq \mathcal{CR}$ are stored in the subset V, and this will extend the candidate that is used for sampling in Eq. (7). If the subset V is zero-length, a one-dimension random sampling $\{1, \dots, Q\}$ is

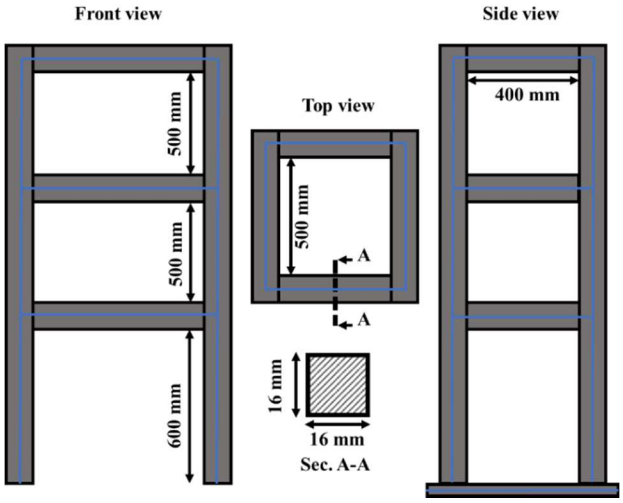

Fig. 1. The three-storey structure

generated to avoid the empty jump vector. The selection crossover probability $P_{\mathcal{CR}}$ is adjusted at every iteration by determining the maximum variation of the N chains and stored in the crossover probability vector $m_{\mathcal{CR}}$. The probabilistic crossover operator enhances the search capability and iteratively proposes new regions in the search space where chains utilize and spread information to extend their current positions.

IV. FEMU Application: Three-Storey Structure

The three storey-structure [15], as shown in Figure 1, is constructed from industrial steel bars, where each bar has a square cross-sectional shape, and all bars are welded together. The structure is anchored to the ground at storey level 1. The elastic modulus is $E = 2.0 \times 10^{11}$ N/m^2, and the material density is $\rho = 7820$ kg/m^3. The cross-sectional area A, the moment of inertia I_x, and the torsional constant J are 2.56×10^{-4} m^2, 5.46×10^{-9} m^4, and 1.09×10^{-8} m^4, respectively.

The structure is excited in the lab using an electromagnetic shaker which is attached at the middle of storey-1. The response is measured by 7 mounted accelerometers, which are distributed at different levels randomly. The measured natural frequencies for the first 6 modes are, $\omega_m = \{62.45, 71.21, 89.64, 205.09, 237.9, 292.01\}$ Hz. To apply the FEMU problem, the structure is modelled through the MATLAB environment, and the FE analysis is performed by the Structural Dynamics Toolbox. Each vertical or horizontal bar is divided into 100 elements.

The three algorithms, the DE-MC, the DE-MCS, and the Pop-MCMC, are tested on this application to update 9 parameters of the FE model. The updating parameters are the elastic modulus E, the material density ρ, and the cross-sectional area A. i.e., three updating elements are assigned for each level, such as E_1, ρ_1, and A_1 are the updating elements for the components (steel bars) of storey-1, which is anchored to the ground. The next two stories, the middle storey-2, and the top storey-3, have the same updating parameters for their components. Thus, the updating parameter vector is $\boldsymbol{\theta} = \{E_1, E_2, E_3, \rho_1, \rho_2, \rho_3, A_1, A_2, A_3\}$. To keep the updated results physically realistic, the uncertain parameters are bounded by maximum and minimum values, which specify the boundary of the search space. Thus, $E_{max} = 2.2 \times 10^{11}$, $\rho_{max} = 8000$, $A_{max} = 2.66 \times 10^{-4}$, $E_{min} = 1.8 \times 10^{11}$, $\rho_{min} = 7700$, and $A_{min} = 2.46 \times 10^{-4}$.

The efficiency of each updating method is investigated by conducting 15 independent runs. Each updating trial is set to $T = 5000$ iterations, and the variance vector $\sigma^2 = \{5 \times 10^{11}, 5 \times 10^{11}, 5 \times 10^{11}, 5 \times 10^{3}, 5 \times 10^{3}, 5 \times 10^{3}, 5 \times 10^{-4}, 5 \times 10^{-4}, 5 \times 10^{-4}\}$. The number of the chains for the DE-MC is $N = 18$, and $\gamma = 2.38/\sqrt{2 \times 9} \approx 0.56$. The DE-MCS parameters are $N_s = 3$, $R_0 = 90$, $\gamma_s = 2.38/\sqrt{2} \approx 1.7$, and $Z = 10$. The number of the chains for the Pop-MCMC $N = 18$, The crossover sequence $\omega_{\mathcal{CR}} = 3$, the number of chain pairs is $e = 3$, and $\gamma = 2.38/\sqrt{2 \times 9} \approx 0.56$.

In general, the algorithms successfully updated the examined structure and produced further accurate updating values. The acquired results are variable since each method is stochastic. Figure 2 presents the real CPU time consumed to implement a complete updating process by each algorithm. The figure shows the computing time in minutes for the 15 independent runs, where the algorithms are tested independently on the same computer device with the same problem set.

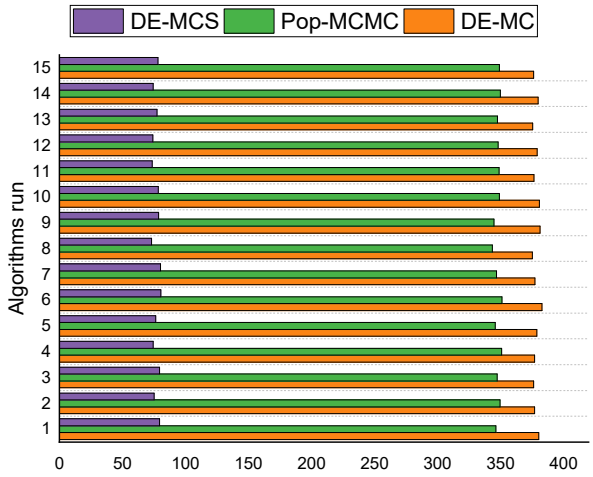

Fig. 2. Real CPU times for the complete updating process

TABLE I. THE UPDATED PARAMETERS OF THE THREE-STOREY STRUCTURE USING DE-MC, DE-MCS, AND POP-MCMC METHODS

	Initial	**DE-MC Method**	**c.o.v (%) $\frac{\sigma_i}{\theta_i}$**	**DE-MCS Method**	**c.o.v (%) $\frac{\sigma_i}{\theta_i}$**	**Pop-MCMC Method**	**c.o.v (%) $\frac{\sigma_i}{\theta_i}$**
E_1	2.0×10^{11}	2.072×10^{11}	2.28	2.110×10^{11}	2.20	2.124×10^{11}	2.00
E_2	2.0×10^{11}	1.863×10^{11}	2.37	1.892×10^{11}	2.10	1.858×10^{11}	1.97
E_3	2.0×10^{11}	1.995×10^{11}	2.40	1.956×10^{11}	2.08	1.962×10^{11}	2.09
ρ_1	7850	7813	2.41	7918.2	2.10	7959.8	1.98
ρ_2	7850	7848	2.25	7929.8	2.30	7927.5	2.35
ρ_3	7850	7824	2.31	7739.2	2.30	7741	2.27
A_1	2.560×10^{-4}	2.585×10^{-4}	2.55	2.629×10^{-4}	2.40	2.638×10^{-4}	2.34
A_2	2.560×10^{-4}	2.565×10^{-4}	2.65	2.548×10^{-4}	2.32	2.581×10^{-4}	2.29
A_3	2.560×10^{-4}	2.535×10^{-4}	2.59	2.489×10^{-4}	2.38	2.482×10^{-4}	2.38

The installed processor is Intel(R) Core (TM) i7-2820QM CPU @ 2.30GHz, and the computer RAM is 16.0 GB. It is clear from the figure that the time needed to complete 5000 iterations is affected by the number of the involved Markov chains ($N = 18$ and $N_s = 3$). The average CPU time of the 15 independent runs for the DE-MC is 387.4 minutes, the Pop-MCMC is 348.0 minutes, and for the DE-MCS is 76.9 minutes. Despite that the DE-MC and the Pop-MCMC algorithms using the same number of chains ($N = 18$), the Pop-MCMC method is around 40

minutes faster than the DE-MC algorithm. This can be explained by the internal determination to propose new solutions by the Pop-MCMC algorithms, where it uses subspace sampling and outliers correction which enhances the convergence speed and thus minimizing the computational demand (time and processing) required for the further searching move. However, the DE-MCS has the lowest CPU time possible since it only utilizes $N_s = 3$ chains.

The updated parameters given by the three algorithms are listed in Table 1. As expected, the algorithms have successfully updated the uncertain parameters and produced physically realistic values. Furthermore, the table includes the coefficient of variation (c.o.v), which is defined as the ratio of the standard deviation σ_i to the mean value of θ_i. The c.o.v values are considered as a standardised measure of dispersion of a probability distribution. Generally, the c.o.v values in Table 1 for all updating parameters by all algorithms reflect an accurate sampling tendency towards the sampled distribution since all values are less than 3.0 %.

Figures 3, 4, and 5 show the boxplots from the DE-MC, the DE-MCS, and the Pop-MCMC algorithms respectively. The plots illustrate the sampling performance of the algorithms for each updating parameter θ_i which are normalized by its initial (mean) value θ_i^0. The median line indicates the resulting updated value for each parameter. As seen in the plots, the algorithms have converged successfully to the high-density intervals, in which the obtained samples are normally distributed around the mean (as the mean and median intersect each other). The outlier samples (red dots) located outside the whiskers of the boxplot (a range within 1.5|QR) are more frequently observed in the samples of the DE-MC algorithm. On the other hand, the Pop-MCMC method has fewer outliers than the DE-MC method. The DE-MCS algorithm has the fewest outliers overall. The fewer outliers demonstrates the advantage of using a smaller number of chains $N_s \ll N$ to obtain sufficient sampling proposals, and thus the DE-MCS updating method is preferable to avoid outlier solutions. Despite that Pop-MCMC uses $N = 18$, the method still reflects significantly smaller outliers than DE-MC, which uses the same number of Markov chains. This is due to the outlier correction step that Pop-MCMC applies at each iteration.

The updated frequencies are given in Table 2. The analytical and the measured natural frequencies are also given. The absolute error of each mode is estimated by $\frac{|f_i^m - f_i|}{f_i^m}$, and the total average error for the six modes is also estimated according to $TAE = \frac{1}{N_m}\sum_{i=1}^{N_m}\frac{|f_i^m - f_i|}{f_i^m}$, $N_m = 6$. As listed in the same table, the updated frequencies by the Pop-MCMC algorithm provide the optimum results among the other algorithms. Pop-MCMC has minimized the TAE from its starting point as 6.58% to 2.30%. The updated frequencies by DE-MCS and DE-MC have better TAE percentages than the initial FE model. DE-MCS and DE-MC have reduced the TAE down to 2.43%, and 2.68% respectively. Overall, the Pop-MCMC algorithm is the best updating technique since it finds the optimum set of solutions. In addition, the method has fewer outliers than DE-MC, even though both use the same number of Markov chains to update the structure. It is worth mentioning here that the DE-MCS

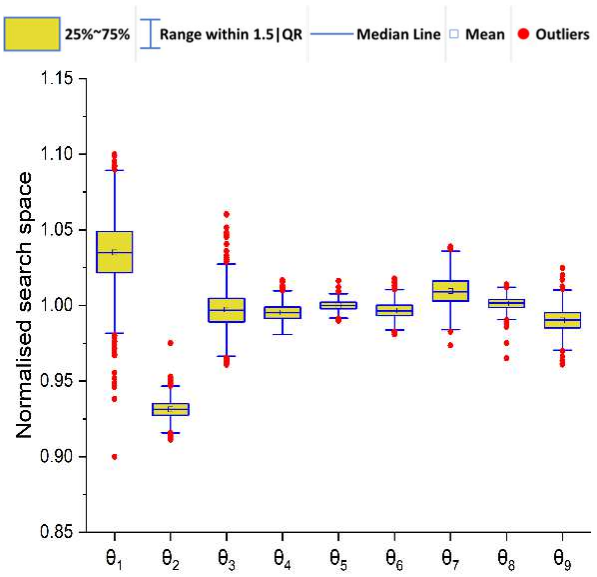

Fig. 3. The boxplots of the normalised updating parameters θ_i/θ_i^0 using the DE-MC algorithm

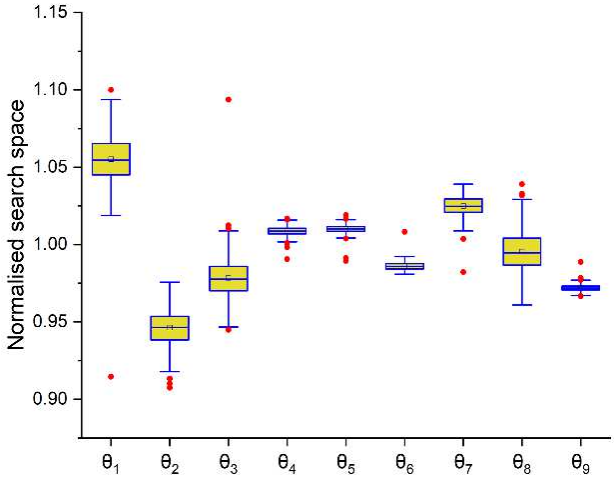

Fig. 4. The boxplots of the normalised updating parameters θ_i/θ_i^0 using the DE-MCS algorithm

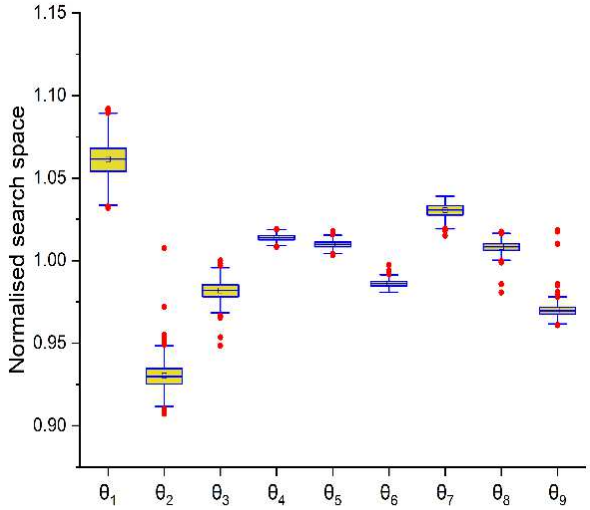

Fig. 5. The boxplots of the normalised updating parameters θ_i/θ_i^0 using the Pop-MCMC algorithm

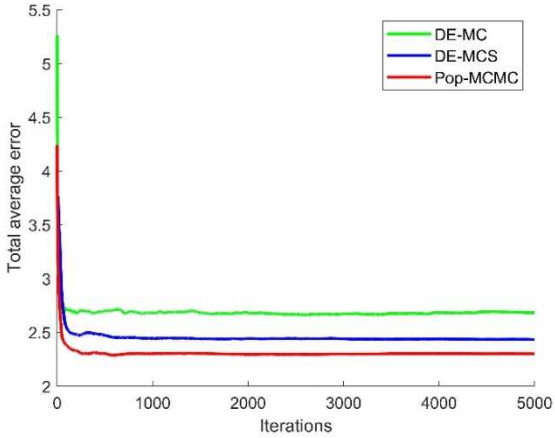

Fig. 6. Evaluation of the total average error of the three-storey structure using the DE-MC, DE-MCS, and Pop-MCMC algorithms

method reflects a promising ability from different perspectives. The real CPU time processing is the lowest, the outliers are the fewest, and the final TAE is significantly reduced.

Figure 6 plots the evaluation of the total average error for the 5000 iterations. It shows that all algorithms started to converge before the first 1000 iterations. However, different starting points (initial total average error) are noticed, which indicates the impact of the genetic operators used to evolve the population (Markov chains). The DE-MC method uses two random chains to generate the future sample. Thus, the information for deep algorithm search is supported by random moves from the current interactive population. Despite this, the random selection is evolved from the past chain experience, which keeps the DE-MC procedure as advanced as the other sampling methods. The Pop-MCMC is better at its starting point than the DE-MC, while the DE-MCS is the best.

TABLE II. THE UPDATED NATURAL FREQUENCIES (HZ) OF THE THREE-STOREY STRUCTURE USING DE-MC, DE-MCS, AND POP-MCMC METHODS

Modes	**Measured freq.**	**Initial freq. (Error %)**	**DE-MC freq. (Error %)**	**DE-MCS freq. (Error %)**	**Pop-MCMC freq. (Error %)**
1	62.45	71.07 (13.8%)	66.58 (6.62%)	65.89 (5.50%)	65.94 (5.59%)
2	71.21	75.40 (5.89%)	71.28 (0.10%)	70.60 (0.85%)	70.79 (0.58%)
3	89.64	91.60 (2.19%)	86.56 (3.44%)	87.60 (2.28%)	87.52 (2.36%)
4	205.09	211.26 (3.01%)	198.07 (3.42%)	196.51 (7.18%)	196.95 (3.97%)
5	237.9	258.82 (8.79%)	241.87 (1.67%)	240.85 (1.24%)	240.80 (1.22%)
6	292.01	308.94 (5.80%)	294.48 (0.85%)	292.50 (0.51%)	292.33 (0.11%)
TAE	-	**(6.58%)**	**(2.68%)**	**(2.43%)**	**(2.30%)**

V. CONCLUSION

This paper compares three advanced multichain methods for the Bayesian model updating problem. Three evolutionary MCMC algorithms are tested by updating a three-storey structural system with real data. The updating problem involved 9 updating parameters, which increase the complexity of the posterior PDF. However, the methods have successfully updated the structural system and produced more accurate updating parameters. It was found that the DE-MCS method requires the lowest computational time possible to estimate the updating problem. This is expected since the algorithm uses only three Markov chains as the default algorithm set. On the other hand, DE-MC needs the most real-time processing and Pop-MCMC is in the middle. The results show that the outliers happen more frequently in the DE-MC procedures. Furthermore, fewer outliers are seen for Pop-MCMC, since it uses an outlier correction step. The lowest outlier count is found in DE-MCS, where only a few chains are adopted in the updating problem. Finally, Pop-MCMC has decreased the initial error of the problem to the lowest possible value. The algorithm is found to be the most accurate search/exploit procedure among the methods considered.

REFERENCES

[1] S. S. Rao, The Finite Element Method in Engineering. Butterworth-Heinemann, 2017.

[2] M. I. Friswell and J. E. Mottershead, Finite Element Model Updating in Structural Dynamics. Springer Science & Business Media, 2013.

[3] I. Boulkaibet, "Finite element model updating using Markov Chain Monte Carlo techniques," University of Johannesburg, 2014.

[4] E. Simoen, G. De Roeck, and G. Lombaert, "Dealing with uncertainty in model updating for damage assessment: A review," Mechanical Systems and Signal Processing, vol. 56, pp. 123-149, 2015.

[5] C. J. Geyer, "Practical Markov Chain Monte Carlo," *Statistical science,* pp. 473-483, 1992.

[6] T. Back, Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms. Oxford University Press, 1996.

[7] I. Boulkaibet, T. Marwala, L. Mthembu, M. I. Friswell, and S. Adhikari, "Sampling techniques in Bayesian finite element model updating," in *Topics in Model Validation and Uncertainty Quantification, Volume 4*: Springer, 2012, pp. 75-83.

[8] R. Dearden, N. Friedman, and D. Andre, "Model-based Bayesian exploration," 2013.

[9] C. J. Ter Braak, "A Markov Chain Monte Carlo version of the genetic algorithm Differential Evolution: easy Bayesian computing for real parameter spaces," Statistics and Computing, vol. 16, no. 3, pp. 239-249, 2006.

[10] M. Sherri, I. Boulkaibet, T. Marwala, and M. I. Friswell, "A differential evolution Markov chain Monte Carlo algorithm for Bayesian model updating," in Special Topics in Structural Dynamics, Volume 5: Springer, 2019, pp. 115-125.

[11] C. J. ter Braak and J. A. Vrugt, "Differential evolution Markov chain with snooker updater and fewer chains," Statistics and Computing, vol. 18, no. 4, pp. 435-446, 2008.

[12] K. B. Laskey and J. W. Myers, "Population Markov chain Monte Carlo," Machine Learning, vol. 50, no. 1-2, pp. 175-196, 2003.

[13] J. A. Vrugt, "Markov chain Monte Carlo simulation using the DREAM software package: Theory, concepts, and MATLAB implementation," Environmental Modelling & Software, vol. 75, pp. 273-316, 2016.

[14] M. Sherri, I. Boulkaibet, T. Marwala, and M. I. Friswell, "Bayesian Finite Element Model Updating Using a Population Markov Chain Monte Carlo Algorithm," in Special Topics in Structural Dynamics & Experimental Techniques, Volume 5: Springer, 2020, pp. 259-269.

[15] P. Alimouri, S. Moradi, and R. Chinipardaz, "Updating finite element model using stochastic subspace identification method and bees optimization algorithm," Latin American Journal of Solids and Structures, vol. 15, 2018.